MASTER ROADMAP / NOVEMBER 2026

AI, Systems, and Cybersecurity

From first principles to reliable, secure AI systems

31 learning stages | Practical projects | Implementations from scratch | Mastery checks

Starting point: Absolute beginner.
Goal: Become an independent engineer who can understand, implement, evaluate, deploy, and secure AI systems, and pursue advanced research in a chosen area.

This curriculum covers the major foundations and branches of AI and cybersecurity. Deep expertise develops through sustained projects and specialization. A topic list alone is not a guarantee of mastery.

Learning method

Understand the problem. Explain the mechanism. Implement a small version. Verify it. Experiment. Apply it. Measure its limitations.

Prepared from the full master roadmap in this conversation. Planning estimates are illustrative, not guaranteed completion times.

NAVIGATION

Contents

03 How to use the roadmap

04 Part I - Computing and software foundations / Stages 0-3

06 Part II - Mathematics and data / Stages 4-5

07 Part III - Machine learning and AI foundations / Stages 6-11

10 Part IV - Language models and AI applications / Stages 12-18

13 Part V - Reliable AI systems / Stages 19-20

14 Part VI - Cybersecurity / Stages 21-25

17 Part VII - AI security and AI for cybersecurity / Stages 26-28

18 Part VIII - Research and integration / Stages 29-30

20 Dependency and sequencing guide

21 Required implementation portfolio

22 Mastery standard and weekly study system

23 Time, hardware, and portfolio expectations

24 First four weeks and reference library

Reading this document
Stages are numbered 0 through 30. Stage pages group two related stages when space permits. The capstone stage has its own page. Numbering organizes the curriculum; the dependency guide explains where stages may overlap.

Security practice
Practice cybersecurity exercises on your own systems, isolated labs, or explicitly authorized targets. Use synthetic or authorized data for security experiments.

METHOD

How to follow this roadmap

The learning cycle
For each concept: understand the problem; explain inputs, outputs, assumptions, and internal steps; study the mathematics; implement a small version; verify correctness; predict and run experiments; use a mature implementation; measure limitations; and apply it in a useful project. For nonmathematical subjects, use protocols, state diagrams, or threat models instead of equations.

1. Mechanism
Work through the algorithm or protocol manually using a tiny example.

2. Educational implementation
Implement the core idea with minimal dependencies.

3. Engineering implementation
Add validation, tests, interfaces, persistence, and failure handling.

4. Production application
Use established components, measure them, and understand their boundaries.

Examples
Implement backpropagation manually, then use automatic differentiation. Implement exact vector search before using a search index. Build a small language model before studying large-scale training. Implement cryptographic examples to understand them; use vetted cryptographic libraries in deployed systems.

Continuous requirements
Begin version control, documentation, testing, debugging, evaluation, error analysis, privacy, access control, secret management, reproducibility, and data provenance immediately. Deepen them throughout the curriculum. Read technical documentation and write clear explanations.

Scope and depth
Aim for implementation depth across the core, engineering depth in your main projects, and research depth in selected specialties. Keep one primary learning stage and one related project active at a time.

PART I - COMPUTING AND SOFTWARE FOUNDATIONS

Stage 0 - Computer and development fundamentals

Understand
Binary, hexadecimal, text encoding, Unicode; CPU, RAM, storage, GPUs, and the memory hierarchy; programs, processes, threads, files, and permissions; operating systems and virtual machines; terminal navigation, environment variables, package installation; Git commits, branches, merges, and conflict resolution; basic internet architecture.

Build from scratch
A command-line file organizer, a file metadata inspector, and a directory-processing script that produces a report.

Project
A reproducible development workspace with installation instructions and a version-controlled project.

Mastery check
Start from a clean environment, install dependencies, run the application, diagnose a failure, and restore a previous working version.

PART I - COMPUTING AND SOFTWARE FOUNDATIONS

Stage 1 - Python and computational thinking

Understand
Types, variables, control flow, functions; lists, dictionaries, sets, tuples, iterators; mutability, references, scope, exceptions; modules, packages, environments, dependencies; classes and composition; type hints, logging, testing, profiling, debugging; files, CSV, JSON, regular expressions, HTTP requests; async programming and concurrency fundamentals.

Build from scratch
A CSV parser for a deliberately limited format, a command-line task manager, a log parser and summary generator, and a simple socket-based HTTP client after networking basics.

Project
A tested data-processing application with useful error messages and a command-line interface.

Mastery check
Handle malformed files, missing values, failed requests, and unexpected inputs without silently producing incorrect results.

PART I - COMPUTING AND SOFTWARE FOUNDATIONS

Stage 2 - Data structures, algorithms, and broader programming

Understand
Time and memory complexity, Big-O; arrays, linked lists, stacks, queues, hash tables; trees, heaps, graphs, tries; searching, sorting, recursion, divide-and-conquer; greedy algorithms, dynamic programming; breadth-first and depth-first search, shortest paths; floating-point representation and numerical errors. Learn enough C for pointers, allocation, stack/heap, compilation, and memory errors. Add basic HTML, CSS, and JavaScript for usable interfaces.

Build from scratch
A hash table, binary heap, graph traversal toolkit, small inverted search index, and LRU cache.

Project
A small searchable collection with an interface and measured algorithm choices.

Mastery check
Choose appropriate structures, justify their complexity, and explain why a numerically or algorithmically naive implementation fails.

PART I - COMPUTING AND SOFTWARE FOUNDATIONS

Stage 3 - Operating systems, networking, and databases

Understand
Scheduling, synchronization, locks, races, deadlocks; virtual memory, file systems; TCP/IP, UDP, DNS, HTTP, TLS, sockets; request lifecycles, proxies, timeouts, retries; relational modeling, SQL, joins, indexes, transactions; isolation, consistency, backups, recovery; caches, queues, object storage; containers and isolation boundaries.

Build from scratch
A local client/server program, small HTTP server, persistent key-value store, queue with retries, and database-backed API.

Project
A small web service with database persistence, authentication, logs, tests, and backup restoration.

Mastery check
Trace a request from client to database, diagnose a slow query, reproduce a race condition, and restore data from backup.

PART II - MATHEMATICS AND DATA

Stage 4 - Mathematics for AI

Understand
Linear algebra: vectors, matrices, products, norms, projections, rank, eigenvalues/eigenvectors, SVD, positive semidefinite matrices, PCA. Calculus and optimization: derivatives, gradients, chain rule, Jacobians, Hessians, constraints, conditioning, convexity, numerical optimization. Probability: conditional probability, Bayes, independence, distributions, expectation, variance, covariance, likelihood, entropy, KL divergence. Statistics: sampling, estimation, uncertainty intervals, hypothesis tests, effect sizes, bootstrap, experiment design, confounding, multiple comparisons. Discrete math: logic, sets, combinatorics, graphs, recurrences.

Build from scratch
Small vector/matrix operations, gradient descent, numerical gradient checking, linear regression, PCA using a numerical SVD routine, and bootstrap uncertainty estimation.

Project
Code-linked mathematical notebooks with numerical examples and experiments. Study alongside applications and revisit advanced material when needed.

Mastery check
Derive linear-regression gradients, explain PCA geometrically, apply Bayes, and distinguish statistical significance from practical usefulness.

PART II - MATHEMATICS AND DATA

Stage 5 - Data engineering and analytical practice

Understand
Collection and provenance; schemas, validation, deduplication, labeling; missing data, outliers, imbalance, sampling bias; exploratory analysis and visualization; batch/streaming pipelines; dataset versioning and reproducible transformations; licensing, consent, retention, deletion; entity resolution and train/test contamination.

Build from scratch
A data validator, reproducible ingestion/cleaning pipeline, dataset quality report, and transformation pipeline with failure recovery.

Project
Turn a genuinely messy dataset into a documented, versioned analytical dataset.

Mastery check
Explain major transformations, identify possible leakage, and regenerate the dataset from its source.

PART III - MACHINE LEARNING AND AI FOUNDATIONS

Stage 6 - Classical machine learning

Understand
Supervised, unsupervised, semi-supervised learning; linear/logistic regression; regularization, scaling; decision trees, random forests, boosting; nearest neighbors, naive Bayes, SVMs; clustering, dimensionality reduction, anomaly detection; calibration, thresholds; cross-validation, hyperparameter search; grouped/temporal evaluation; interpretability and error analysis.

Build from scratch
Linear/logistic regression, k-means, k-nearest neighbors, a small decision tree, and a cross-validation routine.

Project
A real prediction problem with a simple baseline, reproducible preprocessing, justified metrics, and an error report.

Mastery check
Explain why validation reflects deployment. Show failures and whether improvement is meaningful. A negative result is acceptable when the experiment is valid.

PART III - MACHINE LEARNING AND AI FOUNDATIONS

Stage 7 - Broader AI: search, reasoning, and uncertainty

Understand
State-space and heuristic search; A*; constraint satisfaction; propositional/predicate logic; rule-based systems, knowledge representation; planning and scheduling; Bayesian networks, probabilistic inference; decision theory; causal inference foundations including interventions, confounding, and causal graphs.

Build from scratch
An A* pathfinder, small constraint solver, rule-based inference engine, and exact inference for a tiny Bayesian network.

Project
Solve a scheduling/planning problem and compare a deterministic solution with an LLM-assisted approach.

Mastery check
Explain which parts need search, rules, probability, or learned predictions, and why.

PART III - MACHINE LEARNING AND AI FOUNDATIONS

Stage 8 - Deep learning foundations

Understand
Tensors, computation graphs, automatic differentiation; forward/backpropagation; activations, losses, initialization; SGD, momentum, Adam; regularization, dropout, normalization; batching, schedules, checkpoints; gradient instability, precision; generalization and optimization behavior.

Build from scratch
A NumPy multilayer perceptron, manual backpropagation, small scalar autodiff engine, SGD and Adam, and the same model in PyTorch.

Project
Train a neural network, compare it with a classical baseline, and document useful and unsuccessful changes.

Mastery check
Check gradients numerically, overfit a tiny batch, diagnose a broken training loop, and reproduce a saved run.

PART III - MACHINE LEARNING AND AI FOUNDATIONS

Stage 9 - Computer vision

Understand
Images as tensors; convolution, padding, stride, pooling; CNNs, residual connections; transfer learning, augmentation; classification, detection, segmentation; OCR, document layout; vision transformers; similarity, self-supervised representations; robustness to lighting, compression, distribution shifts.

Build from scratch
Two-dimensional convolution, a small CNN with your own training loop, intersection-over-union evaluation, and simple image retrieval.

Project
An OCR pipeline or image classifier tested on images outside its training distribution.

Mastery check
Identify whether failures come from labels, preprocessing, representation, or distribution shift.

PART III - MACHINE LEARNING AND AI FOUNDATIONS

Stage 10 - NLP and information retrieval

Understand
Text normalization, tokenization; n-grams, language modeling; bag-of-words, TF-IDF, BM25; word embeddings; RNNs, LSTMs, sequence modeling; classification, tagging, translation, summarization; multilingual data/tokenization; precision, recall, ranking metrics, relevance judgments.

Build from scratch
An n-gram language model, TF-IDF and BM25 retrieval, a small word-embedding model, and simple sequence model.

Project
A searchable document collection and a text classifier.

Mastery check
Compare keyword retrieval with embeddings on labeled queries and explain different failure cases.

PART III - MACHINE LEARNING AND AI FOUNDATIONS

Stage 11 - Transformers

Understand
Queries, keys, values, scaled dot-product attention; multi-head attention; positional representations; residual connections, normalization, feed-forward blocks; encoder, decoder, encoder-decoder architectures; causal/padding masks; objectives and computational complexity.

Build from scratch
Scaled dot-product attention, multi-head attention, a transformer block, and small decoder-only language model. Use tensor operations and automatic differentiation, while implementing the architecture yourself.

Project
A tested transformer implementation and small next-token modeling experiment.

Mastery check
Verify shapes and masks, compare with a trusted implementation, and demonstrate that future tokens cannot leak into causal predictions.

PART IV - LANGUAGE MODELS AND AI APPLICATIONS

Stage 12 - Language-model data, training, and inference

Understand
Byte-level text representation, tokenization; BPE, vocabulary tradeoffs; corpus cleaning, deduplication, contamination; next-token prediction, cross-entropy; training budgets, checkpoints; temperature, top-k, top-p; context limits, KV cache; perplexity, task evaluation; memorization and dataset quality.

Build from scratch
A BPE tokenizer, small GPT-style model, complete training loop, autoregressive generation loop, and basic KV cache.

Project
Train a small model on a documented corpus and report quality, compute use, and limitations.

Mastery check
Explain how text becomes training examples and logits become generated text. A small model teaches mechanisms; competitive foundation models require very different resources.

PART IV - LANGUAGE MODELS AND AI APPLICATIONS

Stage 13 - LLM application engineering

Understand
Model APIs and local inference; prompt construction/examples; structured outputs, schema validation; context management; timeouts, retries, rate limits, fallbacks; evaluation datasets, regression checks; cost, latency; uncertainty and unsupported answers. This stage can begin after competent Python programming while theory continues.

Build from scratch
A minimal model-client wrapper, structured extraction pipeline, evaluation runner, and small web interface.

Project
A useful document extraction or classification application.

Mastery check
Compare changes on fixed evaluation cases and explicitly handle malformed or unsupported outputs.

PART IV - LANGUAGE MODELS AND AI APPLICATIONS

Stage 14 - Fine-tuning and alignment

Understand
Transfer learning, supervised fine-tuning; instruction datasets, chat formats; LoRA, quantized fine-tuning; preference learning, reward models, DPO, RLHF concepts; overfitting, catastrophic forgetting; contamination, data quality; fine-tuning versus retrieval versus prompting.

Build from scratch
A low-rank adapter layer, supervised fine-tuning for a small model, and toy preference-optimization loss.

Project
Fine-tune a manageable model for one narrow task and compare with a strong prompting baseline.

Mastery check
Demonstrate improvements, regressions, resources required, and circumstances where fine-tuning was unnecessary.

PART IV - LANGUAGE MODELS AND AI APPLICATIONS

Stage 15 - Embeddings, search, and RAG

Understand
Dense/sparse/hybrid retrieval; similarity, normalization; chunking, structure; metadata filtering, access-aware retrieval; approximate nearest neighbors; reranking, query transformation; grounded generation, citations; retrieval/answer evaluation; graph retrieval and appropriate use cases.

Build from scratch
Exact cosine search, chunking/indexing pipeline, minimal RAG system, retrieval metrics, citation/evidence checks. Implement a small approximate index later if infrastructure becomes a focus.

Project
A document assistant answering with supporting passages and handling missing evidence.

Mastery check
Evaluate retrieval separately from generation and verify that users cannot retrieve unauthorized documents.

PART IV - LANGUAGE MODELS AND AI APPLICATIONS

Stage 16 - Agents, workflows, and memory

Understand
Deterministic workflows versus model-directed decisions; tool schemas/execution; state machines, planning; permissions, approvals; timeouts, retry limits, cancellation, idempotency; durable execution, recovery; session state, persistent memory; contradictions, expiry, deletion, user isolation; multi-agent coordination and additional failures.

Build from scratch
A bounded tool-use loop, workflow state machine, memory read/update/delete store, execution traces, and replay tests.

Project
An assistant that searches documents, queries a database, and prepares drafts.

Mastery check
Handle tool failure, hostile tool content, and repeated requests. Justify every permission. Add multiple agents only for a measured need.

PART IV - LANGUAGE MODELS AND AI APPLICATIONS

Stage 17 - Generative vision, speech, and multimodal systems

Understand
Autoencoders, VAEs; GAN objectives/instability; diffusion, denoising, conditioning; audio sampling, Fourier transforms, spectrograms; recognition, synthesis, voice activity detection; vision-language models; image/audio/video/document integration; streaming and end-to-end latency.

Build from scratch
A small autoencoder/VAE, toy diffusion training/sampling loop, audio feature pipeline, and multimodal application using pretrained components.

Project
A voice document assistant and image-aware document analysis tool.

Mastery check
Evaluate each component independently and measure complete response latency and failure rates.

PART IV - LANGUAGE MODELS AND AI APPLICATIONS

Stage 18 - Reinforcement learning and control

Understand
MDPs; policies, values, rewards, Bellman equations; dynamic programming, Monte Carlo; temporal differences; Q-learning, DQN, policy gradients, actor-critic; PPO, exploration; reward hacking, shift; offline/model-based RL; partial observability; control/robotics foundations where relevant.

Build from scratch
A grid world, value iteration, Q-learning, small DQN, and policy-gradient agent.

Project
Compare methods in simulated environments using multiple seeds.

Mastery check
Separate algorithm behavior from random variation and explain unintended reward behavior. Robotics adds control, estimation, geometry, and hardware work.

PART V - RELIABLE AI SYSTEMS

Stage 19 - Software architecture, deployment, and MLOps

Understand
API design, service boundaries; authentication, authorization, tenant isolation; containers, deployment, configuration; queues, caching, background jobs; dataset/model versioning; tracking, artifact management; CI/CD, evaluation gates; monitoring, rollback, drift, incidents; reliability targets and recovery.

Build from scratch
A model-serving API, evaluation job, model/prompt release manifest, monitoring and rollback workflow.

Project
A usable deployed AI product with a release process and operational dashboard.

Mastery check
Recover from a bad release, dependency outage, malformed-input burst, and database restoration exercise.

PART V - RELIABLE AI SYSTEMS

Stage 20 - AI performance and distributed systems

Understand
Profiling and bottlenecks; CPU/GPU memory; vectorization, kernels; floating-point formats, quantization; batching, continuous batching; KV-cache memory; data/tensor/pipeline parallelism; distributed communication/failures; speculative decoding; throughput, latency, quality, and cost.

Build from scratch
A benchmark harness, toy quantizer, batched inference service, and small distributed-training exercise when hardware permits.

Project
Optimize a real bottleneck and publish before/after measurements.

Mastery check
Demonstrate improvement under representative load without unacceptable quality loss.

PART VI - CYBERSECURITY

Stage 21 - Security foundations and cryptography

Understand
Assets, threats, vulnerabilities, risks, controls; confidentiality, integrity, availability; trust boundaries, threat modeling; least privilege, defense in depth; authentication versus authorization; hashes, MACs, encryption, signatures, key exchange; password hashing, secure randomness; certificates, PKI, TLS, key lifecycle; logs and secure defaults.

Build from scratch
A threat model, file-integrity checker using a trusted hash implementation, toy cipher with weakness demonstration, and signed-message workflow using vetted libraries.

Project
Secure an earlier service's identity, secrets, transport, and access controls.

Mastery check
Explain why hashing is not encryption, why authentication is insufficient for authorization, and what happens when a key is exposed.

PART VI - CYBERSECURITY

Stage 22 - Web, API, and application security

Understand
Input validation, output encoding; SQL injection, XSS; CSRF, SSRF, path traversal; object-level authorization; sessions, cookies, OAuth, OpenID Connect; file uploads, unsafe deserialization; vulnerable dependencies; business-logic flaws; abuse prevention, limits; secure code review.

Build from scratch
A deliberately vulnerable local application, security regression tests, object-authorization checks, rate limiter, and upload validator.

Project
Find, document, fix, and retest weaknesses in your lab application.

Mastery check
For each finding, explain root cause, impact, remediation, and evidence that the fix works.

PART VI - CYBERSECURITY

Stage 23 - Network, operating-system, and cloud security

Understand
Segmentation, firewalls, remote access; packet captures, protocols; Linux/Windows security; users, groups, permissions, processes, services; hardening, patches; shared responsibility; IAM, temporary credentials; storage/network permissions; container security, orchestration; infrastructure-as-code review; backups and disaster recovery.

Build from scratch
An isolated network lab, configuration-audit script, cloud-permission review exercise, and container hardening checks.

Project
Deploy a small service with restricted connectivity and permissions, then verify its exposure.

Mastery check
Trace which identity can access which resource and explain a compromised component's blast radius.

PART VI - CYBERSECURITY

Stage 24 - Detection, incident response, and forensics

Understand
Endpoint/network/identity/application logs; collection, normalization, correlation; detection rules, false positives; triage; incident preparation, containment, recovery, review; evidence integrity, timelines; disk/memory/network forensics foundations; threat intelligence and behavior-based analysis.

Build from scratch
A log normalization pipeline, rule-based detections, incident timeline generator, and alert-triage dashboard.

Project
Simulate a harmless lab incident, detect it, investigate it, and write a recovery report.

Mastery check
Connect alerts to evidence and explain remaining uncertainty.

PART VI - CYBERSECURITY

Stage 25 - Security testing and advanced specialist branches

Understand
Shared foundation: scope, rules of engagement, authorized asset discovery, vulnerability assessment, manual verification, secure reporting, remediation testing, responsible disclosure. Branches: application security and secure development; penetration testing in controlled labs; detection engineering; digital forensics; reverse engineering with assembly/executable formats/debuggers; malware analysis in isolated environments; cloud security; governance/risk/audit/supplier risk; hardware, IoT, and OT security with embedded systems and operational constraints.

Build from scratch
A scoped assessment plan, verification records, prioritized findings, and remediation retests.

Project
Complete one authorized assessment with reproducible findings and clear fixes.

Mastery check
Explain assessment scope, evidence, severity, remediation, and retest results. Each specialist branch requires sustained practice beyond a survey.

PART VII - AI SECURITY AND AI FOR CYBERSECURITY

Stage 26 - Adversarial machine learning and model privacy

Understand
Attacker goals, knowledge, capabilities; evasion, adversarial examples; poisoning, backdoors; extraction; membership inference, inversion; memorization; differential privacy foundations; federated learning and limitations; robustness evaluation, adaptive attackers.

Build from scratch
A toy adversarial-example experiment, controlled small-dataset poisoning experiment, robustness harness, and simple synthetic-data privacy experiment.

Project
Evaluate a small model under explicitly defined attacker capabilities.

Mastery check
Report clean accuracy, attacked performance, defense tradeoffs, and limits. One successful defensive test does not establish general security.

PART VII - AI SECURITY AND AI FOR CYBERSECURITY

Stage 27 - LLM, RAG, agent, and multimodal security

Understand
Direct/indirect injection; untrusted documents/retrieved instructions; information disclosure; unsafe generated-output handling; tool misuse, excessive permissions; retrieval poisoning, cross-user leakage; memory poisoning; model/dependency supply chain; resource exhaustion and spending abuse; multimodal injection; evaluation manipulation; filter and judge limitations.

Build from scratch
A local adversarial corpus, tool authorization layer, tenant-aware retrieval checks, constrained execution environment, and tool/data-access audit trails.

Project
Assess and harden your own document assistant.

Mastery check
Show that hostile retrieved content cannot grant permissions, alter authorization, or expose another user's documents. Treat model outputs and tool arguments as untrusted. Enforce permissions in code and measure useful task completion alongside security.

PART VII - AI SECURITY AND AI FOR CYBERSECURITY

Stage 28 - AI-assisted cybersecurity

Understand
Log classification; alert grouping/prioritization; phishing classification; anomaly detection; threat-intelligence extraction; secure-code assistance; retrieval over incidents/playbooks; human oversight and automation limits.

Build from scratch
A classical security-classification baseline, temporal evaluation pipeline, security-document retrieval assistant, and constrained alert-summary workflow.

Project
A security analyst assistant that summarizes evidence, cites sources, and proposes next steps.

Mastery check
Measure precision, recall, false positives, analyst effort, and failure consequences. Account for imbalance, environmental change, and leakage; benchmark accuracy alone is insufficient.

PART VIII - RESEARCH AND INTEGRATION

Stage 29 - Research methods and advanced evaluation

Understand
Begin small exercises after Stage 6. Deepen literature search, paper reading; research questions, falsifiable hypotheses; baselines, controls, ablations; dataset selection, contamination; uncertainty, multiple seeds; human evaluation, rater agreement; model-judge bias; reproducibility; negative results, limitations; technical writing and peer review.

Build from scratch
A reusable experiment runner, versioned evaluations, automated result tables with uncertainty, and reproduction package.

Project
Reproduce one bounded published result, then investigate a meaningful variation.

Mastery check
Another person can reproduce the main result and your conclusions stay within the evidence.

Stage 30 - Integrated capstones

Complete these incrementally. Reuse tested components from earlier stages and integrate one capability at a time.

Capstone A - A reliable predictive system
Include documented provenance, reproducible preprocessing, a simple baseline and justified model, valid evaluation, deployment, monitoring, access controls, and recovery procedures. Evidence: results, error analysis, operational measurements, and a threat model.

Capstone B - A secure personal AI assistant
Build in order: text interface; document ingestion; search with citations; evaluation and unsupported-answer handling; one permission-controlled tool; persistent preferences with update/deletion; authentication and user isolation; monitoring, spending limits, recovery; voice input/output; image/document understanding when useful. Evidence: task evaluations, security tests, latency/cost, failure traces, and permission model.

Capstone C - An AI security laboratory
Include small models, isolated applications, controlled adversarial datasets, injection/retrieval-poisoning cases, privacy/authorization tests, attack/defense comparisons, and reproducible reports. Evidence: defined attacker capabilities, outcomes, mitigations, and residual risks.

Capstone D - An AI-assisted security operations tool
Include synthetic or authorized telemetry, rule-based and ML baselines, evidence-linked summaries, human review, false-positive analysis, and safe workflow boundaries. Evidence: detection quality, operational usefulness, and an incident case study.

PLANNING

Dependency and sequencing guide

Main foundations
Computing -> Python -> algorithms -> systems and databases.

Model understanding
Math + data -> classical ML -> deep learning -> transformers -> language-model training -> fine-tuning.

Application engineering
Python + HTTP + testing -> simple LLM application -> retrieval -> workflows -> deployment.

Cybersecurity
Systems + networking -> security foundations -> application/cloud security -> detection and authorized assessment.

AI security
Security foundations + relevant AI mechanisms -> adversarial ML and LLM security -> AI security capstone.

Specialist sequencing
Computer vision follows deep learning. NLP retrieval can begin before deep learning. Speech and diffusion follow their relevant mathematical and neural foundations. RL follows probability, optimization, and basic ML. Distributed optimization follows a working single-machine implementation.

Parallel learning
Security, evaluation, documentation, and privacy run throughout. Application projects may start before advanced model training. Keep one primary stage and one related project active; do not attempt all branches simultaneously.

Decision points
After foundations, choose a primary focus: applied AI, ML engineering, model research, AI infrastructure, or a cybersecurity role. Preserve broad literacy while taking your selected direction to engineering and research depth.

IMPLEMENTATION INVENTORY

Required implementation portfolio

Computing and software
Hash table, heap, graph traversal, cache; HTTP client/server; persistent key-value store; queue and retries; database-backed service.

Mathematics and machine learning
Gradient descent, numerical gradient checks; linear/logistic regression; k-means, KNN, small decision tree; cross-validation and bootstrap evaluation.

Deep learning and language models
MLP, manual backpropagation; small autodiff engine; convolution, training loops; attention, transformer blocks; BPE, small GPT; sampling, KV caching; low-rank adapter.

Retrieval and agents
Inverted index, TF-IDF, BM25; exact embedding search; RAG; bounded tool loop, workflow engine; memory update/deletion; evaluation runner.

Broader AI
A* search, constraint solver; small probabilistic inference example; autoencoder/VAE, toy diffusion; Q-learning and one neural RL algorithm.

Security
Threat models, access-control tests; integrity checking, log processing; detection rules, incident timeline; controlled robustness/poisoning experiments; permission enforcement and adversarial test harness.

Implementation boundaries
Small educational implementations reveal mechanisms. Production systems need maintained libraries, validated configurations, operational controls, and review. Correctness and useful evidence matter more than the number of repositories.

PROGRESS AND HABITS

Mastery standard and weekly study system

Four levels of understanding
1. Literacy: recognize and explain. 2. Implementation: build a small correct version. 3. Engineering: apply under realistic constraints. 4. Research: analyze assumptions, reproduce results, investigate improvements.

Evidence for each major stage
Explain in your own words; produce a correct implementation and meaningful tests; compare with a baseline or trusted reference; analyze failures; complete a transfer task on unfamiliar data or requirements; write a short report. Do not require timed memorization of large implementations.

Weekly allocation
Implementation/debugging: 40%. Theory/mathematics: 25%. Experiments/evaluation: 20%. Recall/documentation/review: 15%. Adjust to the stage and what your errors reveal.

Every session
Recall a previous concept. Study one bounded idea. Implement or experiment. Record what worked, what failed, and the next question.

Every week
Finish one useful increment. Revisit an earlier concept. Write one explanation without copying. Review one failure carefully.

Every month
Demonstrate a working project. Solve an unfamiliar transfer task. Refactor a weak component. Update your skills and project backlog.

Using AI assistance
Use AI for explanations, critiques, and debugging guidance. Regularly explain and rebuild core mechanisms without generated solutions. Verify technical claims and understand any code you retain.

Time, hardware, and portfolio expectations

Planning estimates
Shared computing, math, data, and security foundations: 700-1,100 focused hours. Core AI implementations and useful applications: another 800-1,200 hours. Broader branches, cybersecurity depth, and capstones: another 1,000-1,700 hours. Total: roughly 2,500-4,000 hours, with projects partly overlapping stages. These are illustrative estimates, not measured guarantees.

Calendar expectations
At 15 focused hours weekly, this is approximately 3-5 years before interruptions. Useful products and narrower job-relevant competence can come much earlier. No fixed deadline establishes expertise across both fields.

Hardware progression
Begin with your existing computer. Use CPUs, small datasets, and small models for foundations. Rent or share GPUs for bounded experiments. Set spending limits before training. Add distributed infrastructure after understanding single-machine bottlenecks. A well-designed small experiment often teaches more than an expensive run.

Portfolio collections
1. Foundations: algorithms, systems, math, classical ML. 2. Model mechanisms: neural nets, transformers, language models, broader AI. 3. Applications: retrieval, workflows, memory, multimodal systems. 4. Cybersecurity: secure apps, detection, incidents. 5. AI security: adversarial evaluation, privacy, controls. 6. Capstones/research: products and reproducible reports.

Each substantial project includes
Problem and intended user; architecture and assumptions; data sources and permissions; setup/reproduction instructions; baselines/evaluation; results/failure cases; security/privacy; cost/performance; known limitations and next experiments.

GETTING STARTED

First four weeks and reference library

Week 1 - Environment and Python
Set up Python and Git. Learn variables, conditions, loops, functions, and file operations. Deliverable: a small command-line utility.

Week 2 - Structures and files
Learn lists, dictionaries, CSV/JSON, exceptions, and simple tests. Deliverable: a file-processing tool that handles invalid inputs.

Week 3 - HTTP, SQL, debugging
Use an API, save selected data in SQLite, and add logging. Deliverable: a data collector with useful failure messages.

Week 4 - Complete a small product
Add documentation, tests, and a simple interface. Rebuild an important component independently. Deliverable: a usable, reproducible project and short demonstration. Use these deliverables as placement checks if already experienced.

Verified reference starting points

Stanford CS336 - advanced language modeling from scratch

NIST NICE - cybersecurity tasks, knowledge, and skills

OWASP - web application security risks

OWASP - LLM application security

MITRE ATLAS - adversarial threats to AI systems

Stanford CS336 assumes Python, machine/deep learning, mathematics, and systems knowledge. Use it after the foundations. NIST NICE supports competency planning; OWASP and MITRE support threat and security-test coverage. Recheck evolving guidance when you reach the relevant stage.